Thread Local

22 January 2022 12:14

USE CASE 1: To create Per-thread instances for memory efficiency and thread-safety

```java
public class UserService {

    public static void main(String[] args) throws InterruptedException {

        for (int i = 0; i < 10; i++) {
            int id = i;
            new Thread(() -> {
                String birthDate = new UserService2().birthDate(id);
                System.out.println(birthDate);
            }).start();
        }

        Thread.sleep(1000);
    }

    public String birthDate(int userId) {
        Date birthDate = birthDateFromDB(userId);
        SimpleDateFormat df = new SimpleDateFormat("yyyy-MM-dd");
        return df.format(birthDate);
    }

    // ......
}
```

- Consider this flow. We replace threads with ExecutorService with a fixed threadpool as our threads grow to 1000s.
- Each thread creates its own SimpleDateFormat objects.
- For 1000 tasks, we have 1000 SimpleDateFormat objects.
  But this is inefficient, as all threads can use the same one.

```java
private static ExecutorService threadPool = Executors.newFixedThreadPool(10);
private static SimpleDateFormat df = new SimpleDateFormat("yyyy-MM-dd");

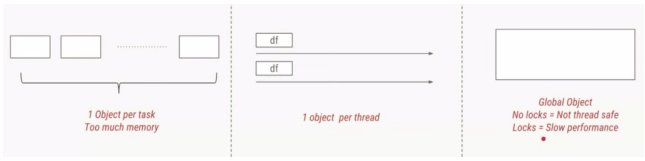
public static void main(String[] args) throws InterruptedException {

    for (int i = 0; i < 1000; i++) {
        int id = i;
        threadPool.submit(() -> {
            String birthDate = new UserService2().birthDate(id);
            System.out.println(birthDate);
        });
    }

    Thread.sleep(1000);
}

public String birthDate(int userId) {
    Date birthDate = birthDateFromDB(userId);
    return df.format(birthDate);
}
```

- This access, though, of this static object, is not thread safe. SimpleDateFormat is not a thread-safe class. It can have data integrity issues
- We can use locks to ensure exclusive access. But this synchronization will slow down our class and code a lot.



- We instead want this middle ground, where each thread has it's own object. Not each task, but each thread. This way, 1000 tasks and 10 threads creates 10 objects, and not 1000.
- This concept of having variable specific to a particular thread is called ThreadLocal
- Since each thread only accesses it's own object, access is thread safe.

```java
class ThreadSafeFormatter {

    public static ThreadLocal<SimpleDateFormat> dateFormatter = new ThreadLocal<SimpleDateFormat>(){
        @Override
        protected SimpleDateFormat initialValue() {
            return new SimpleDateFormat("yyyy-MM-dd");
        }
        @Override
        public SimpleDateFormat get() {
            return super.get();
        }
    };
}

public class UserService {

    public static void main(String[] args) {
        // ....
    }

    public String birthDate(int userId) {
        Date birthDate = birthDateFromDB(userId);
        final SimpleDateFormat df = ThreadSafeFormatter.dateFormatter.get();
        return df.format(birthDate);
    }
}
```

Called once for each thread

1st call = initialValue()
Subsequent calls will return same initialized value

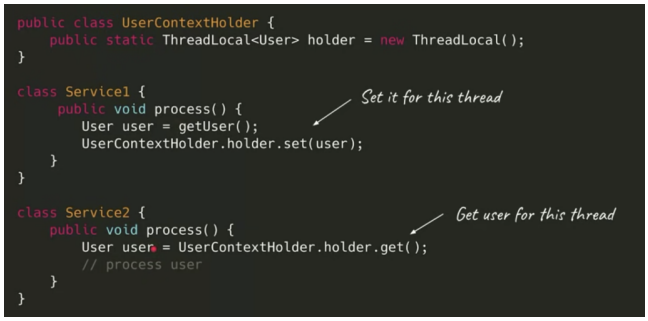
Each thread will get its own copy

USE CASE 2: To ensure a context for a particular thread. All classes involved in the flow of a particular thread use the same context

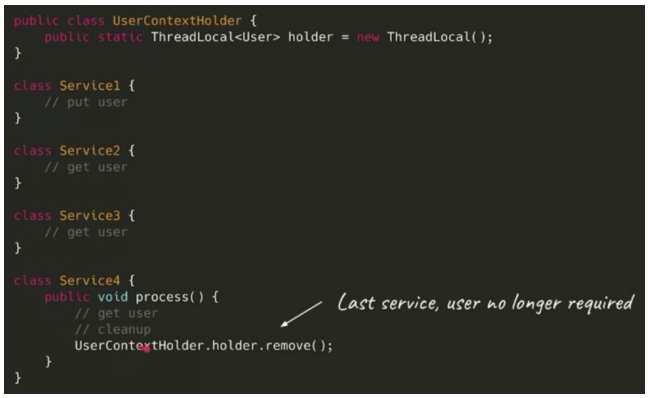
This is done to

1. Avoid passing the same context to all classes.

2. Avoid having to use synchronization, that would add overhead.



IMPORTANT
These threads process requests. We need to ensure once processing of request is over, we must clear the ThreadLocal object.
So the last class that the flow passes through, must clear the request attributes that it stored once the processing is done.



OVERALL USE CASES

- Using one object per-thread, guaranteeing thread confinement (thread safety), because we don't use global object.
- We can use it to store context in relation to a request flow

TO REMEMBER WHEN USING

- Always remember to clean up threadlocal once processing is done, and you're out of scope
- Try using local variables wherever possible so threadlocal isn't required
- Delegate creating and management of threadlocal to frameworks as much as you can so you don't end up making mistakes

Questions/Answers

Ques: **How about making the SDF Final? Also if the SDF itself is not changing I did not understand how it impacts the other objects like Calendar?**
Ans: Final just prevents reassignment of the reference. The problem is that SDF is not immutable - its state changes when it is used to format a date.
If SDF only had global state variable, set only once during Constructor, we would not have thread-safety problem. Unfortunately, multiple global variables (eg: calendar) are modified even when format() method is called. Thus if 2 threads use same SDF object, and both call format() at the same time, one thread's format() will call calendar.setTime(date) and then other thread will do the same overriding first thread's date.

Ques: We said having task local variables (which can be reused) is not good because it creates a lot of extra objects. But wouldn't the extra objects get created only when the task is actually executed by the thread. And also once the task is done, the object will be destroyed ? In that case wouldn't we still be using 10 objects corresponding to the 10 threads at any given point in time?
Ans: Task objects are created when we call new , so even if they are executed late by thread they still occupy heap. Also, even though object is garbage collected once thread executes the task is still creating and garbage collecting N objects as N tasks (which could be 1000s) instead of fixed number like thread local.